Shaikh Zeeshan Murshed

Senior Frontend Engineer | **React** | **TypeScript** | **Full-Stack**

Montréal, QC, Canada | Open to Remote | sk.zeeshan1992@gmail.com | linkedin.com/in/skizsk | github.com/sk-izsk | izsk.dev | izsk.hashnode.dev

Languages: English, Bengali, Hindi

Summary

Senior frontend engineer with 7+ years shipping React and TypeScript products for enterprise SaaS, now extending into Node, React Native and AI-assisted engineering.

Technical Skills

Languages: JavaScript (ES2022+), TypeScript, Python, HTML5, CSS3
Frameworks: React, Next.js (3+ years: Pages Router, then App Router; SSR/SSG, server actions, middleware), Node.js, Express, FastAPI, GraphQL (Apollo Client, urql, Codegen), React Native (Expo), micro-frontends (single-spa)
Web Platform: Service Workers (offline, caching, web push, Background Sync), Web Workers, list virtualization, code splitting, WCAG AA accessibility
State / Data: TanStack Query (React Query), Redux Toolkit, Zustand, Zod
UI Libraries: Material-UI (MUI), Ant Design, Tailwind CSS, Shadcn, NativeWind, Styled-Components, Bootstrap, JSS
Testing: Jest, Vitest, Cypress (E2E), React Testing Library, React Native Testing Library, jest-axe, Enzyme, Mocha, Chai, Nock
Tools / DevOps: Git, GitHub, GitHub Actions, GitLab CI/CD, Docker, Webpack, Vite, Storybook, Husky, Knip, Postman, Figma, Jira, Notion, Bun
Visualization / 3D: Three.js, React Three Fiber, Drei, Mafs, Framer Motion
Real-Time / Push: Socket.IO, WebSockets, Server-Sent Events (SSE), push notifications (Expo Notifications, APNs, FCM)
Analytics / Monitoring: Google Analytics, Mixpanel, Sentry (React Native and web), Core Web Vitals
Databases: PostgreSQL (Neon), MongoDB, Drizzle ORM (versioned migrations), Upstash Redis
Auth / Cloud: Clerk, JWT (custom auth: argon2id, refresh-token rotation), SSO (OAuth, OIDC, SAML; Google, Microsoft, Okta), Stripe, Vercel, AWS (S3, Lambda), Cloudinary, Heroku, Netlify, Render
AI & Agentic Tooling: MCP servers (custom: SDK docs, Figma, agent API; in use: Storybook, GitHub), custom AI skills, Claude Code and Codex agent workflows, AI-assisted code review (GitHub Copilot), prompt engineering, LLM integration (Groq, Anthropic SDK, OpenAI-compatible APIs), RAG and agent workflows

Experience

Applied AI & Full-Stack Engineering Mar 2026 – Present
AI data platforms, product builds, technical writing *Montréal, QC*

- Evaluate and review LLM outputs on AI data platforms, scoring answers for correctness and reasoning quality so model teams get cleaner training and evaluation data.
- Publish a technical post every Monday on advanced React patterns and AI engineering (15+ so far at izsk.hashnode.dev), with working examples readers can reuse.
- Build RAG, agent workflows, FastAPI and Node projects hands-on to deepen my full-stack and AI skills; the larger ones are listed under Projects.

Software Engineer Nov 2020 – Mar 2026
SwiftConnect — Montréal, QC *Enterprise access & security management SaaS platform*

- Replaced copy-pasted API code in every project with one versioned Node.js SDK and a React SDK on top of it, so a backend change became one fix and a version bump. Almost the whole front-end team adopted them, and when the company began selling its API, third-party customers integrated through the same SDKs.
- Published the SDKs from a monorepo (pnpm workspaces, later Bun workspaces) to a private npm registry with semantic versions, a changelog and breaking changes called out. The component library sits in its own repo, with a Webpack config I wrote from scratch, and publishes to a private registry too.

- Built a custom MCP server over the SDK docs so engineers query API structure and usage from their editor instead of going back and forth on documentation.
- Consolidated three projects' copies of the same UI into one component library I built from scratch (roughly two thirds less duplicated code), and a fourth app, moved from Angular 1.6 to React, used it from day one. Components are checked visually and tested in Storybook, with accessibility handled in one place, before any app imports them, so one fix lands everywhere. Teammates started contributing in the same pattern.
- Built a customer-facing meeting-room booking platform in Next.js from scratch (routing, state management, base patterns), chosen for load speed and SEO. It started on the Pages Router and moved to the App Router for the newer APIs, with static pages kept static and the rest rendered on the server, which gave customers faster loads and better SEO. It uses API routes, server actions and middleware, and has run in production with real customers for more than 3 years.
- Connected the component library to Storybook's MCP, so developers pull component usage straight into the editor.
- Cut design revision cycles by roughly 25 to 30% by keeping design rules (padding, borders, colours, shading) in a JSON contract and checking each component against Figma, through a custom Figma MCP and skill, before the PR merges to the SDK. The check says whether a mismatch is a design problem or a code one, so designers only get pulled in when the design itself is wrong.
- Reduced PR turnaround from about 2–3 days to about 1–1.5 days (roughly 50%) by adding a GitHub Copilot first-pass review before a human looks, plus a GitHub MCP that pulls PR comments and status into the editor. Small PRs often approve straight away, and people still make the final call.
- Wrote custom AI skills carrying each project's patterns, boilerplate and Figma-check rules, so generated code starts in the team's style instead of being rewritten in review. Wrote them for several projects, including a front-end pattern skill and a boilerplate skill.
- Built, from scratch, the internal React Native (Expo) dashboard that lets customer support see failures and respond from company phones, with push notifications when a customer's system fails. I was the only engineer with prior React Native experience, set up navigation and API integration on the existing SDK and design system, built the data-heavy screens, and covered them with React Native Testing Library. CPU-heavy calculations on large lists run in background workers (react-native-workers) instead of the main JS thread.
- Delivered a React web dashboard for support, ops and customer teams: its error table shows what failed in access management, and Slack alerts reach their channels, so they stopped contacting engineers to decode raw logs. I built the UI, the push alerts and the error table as part of the team.
- Cut email template turnaround from 3–4 days to hours or minutes by moving SendGrid templates into shared React components that engineers build and CI/CD deploys, which freed designers from hand-written HTML and manual deploys.
- Replaced a daily forced logout with 15-minute access tokens that renew from a session token on user activity or focus. A session token renews up to 3 times before a new one is issued, and long inactivity still logs the user out. I designed it, built the proof of concept, took team feedback and implemented it with the team, so customers stay signed in longer without weakening security.
- Added event tracking (Google Analytics first, then Mixpanel, depending on the project) so the team could see which features customers used, polish those, and retire unused ones to cut maintenance.
- Set up Sentry on the React Native app first, then on the web apps, covering iOS and Android crashes and hangs, slow loads, JS errors, release tracking, source maps, alert routing and the console logs leading up to each error. Core Web Vitals (LCP, CLS) are tracked in production through Sentry and checked locally in Chrome DevTools.
- Replaced React Navigation with Expo's file-based router to cut Dependabot alerts and config overhead. I took the risk case to leadership first and did the migration on a branch without changing behaviour.
- Introduced a three-way kickoff while acting as proxy Product Owner for a few sprints: design walks through the user flow, then backend and front-end agree an API contract in front of the design, and front-end builds on mock data while backend works. Later POs kept the practice and the back and forth between the three teams dropped.
- Mentored 5 or 6 interns and juniors on navigating the project, the team's expectations, long-term product thinking and shipping features on their own; several shipped independently under guidance. I also interviewed junior and

intern candidates in technical rounds.

- Built the core access-control web client in React and TypeScript, with Redux Toolkit and TanStack Query handling complex authorization data so the UI stays stable during heavy mutations across multiple product areas.
- Set up single-spa from scratch and worked with it for more than 3 years to run two shared micro-frontends, an authentication app (login, sign up, forgot password) and a checkout app. The other web apps redirect to them and tokens travel in cookies. I chose this over a shared npm package for independent deploys, team ownership and runtime isolation, and it cut conflicts between teams.
- Integrated SSO sign-in with Google, Microsoft and Okta (OAuth, OIDC, SAML) in the shared auth app, including the token layer that comes from the third-party APIs.
- Built the shared checkout app for room bookings on Stripe and styled it to match our product. Card data stays with Stripe and never touches our code.
- Stopped big forms from re-rendering every field on each keystroke by replacing one form-wide React Context with a Zustand store where each input subscribes only to its own value. The team adopted the pattern for complex forms across most of our web and mobile projects.
- Moved UI state from Redux Toolkit to Zustand across several projects and kept server data in TanStack Query, rather than copying API responses into a store that TanStack Query already caches.
- Pushed back on a read-only, delete-only design for multiple emails and phone numbers, since a typo meant deleting the entry. I built a POC with editable inputs for the PO and designer, and it shipped after customer feedback raised the same problem.
- Added list and table virtualization to heavy tables in the access management app, where the backend returned large lists, and used memoization with React Profiler to stop unrelated components re-rendering across several projects.
- Migrated the Create React App web apps (customized with react-app-rewired) to Vite, with code splitting, dynamic imports, tree-shaking and cleaned-up imports and exports, which roughly halved the bundle.
- Built service worker support for a web app: offline use, cached image assets, FCM web push, and background sync of CSV contact uploads (the Background Sync API first, later our own queue). Moved CSV parsing (files up to 5 MB) and other CPU-heavy work into Web Workers to keep the main thread clear.
- Built keyboard navigation, focus management and colour contrast into the component library to WCAG AA, after a customer reported a focus problem on a button. Checks use the Storybook a11y addon, jest-axe and manual screen reader passes, first in a Husky pre-commit hook and later in CI. We started at AAA and settled on AA when AAA cost too much time.
- Owned the code review and CI standards: GitHub Copilot reviews first, and Knip in CI catches unused exports, imports and dependencies. I wrote RFCs early on, later reviewed other people's, and kept the record of architectural decisions (ADRs).

Full Stack Developer Oct 2020 – Nov 2020
Hatchways — Remote *Agile team of 4*

- Shipped full-stack features in 2-week sprints with a team of four, using React and Material-UI on the front end and Express, Node.js, MongoDB, Cloudinary and JWT authentication on the back end.

Junior Software Developer July 2019 – June 2020
Guiker Inc. — Montréal, QC *Mobile-first apartment rental marketplace startup*

- Cut lease signing from up to a week of agent visits to 1–2 days by building the online lease flow and in-app landlord–tenant chat, which also let landlords and tenants check the lease any time and helped support onboard customers faster.
- Contributed to the React and TypeScript rewrite of the main web product (Material-UI, JSS, REST APIs) with Jest test coverage.
- Owned some Node.js/Express routing and middleware logic, which cut front-end-to-backend integration issues.

- Built the 9-page lease form and fixed needless re-renders caused by React Context with memoization.
- Built mobile features for iOS and Android with React Native (Expo), TypeScript and Jest.

PROJECTS

Tubewise | *Full-Stack Course Platform with an AI Agent API* | tubewise.vercel.app | github.com/sk-izsk/yt-playlist-course

- Built and deployed solo a full-stack app (Next.js 16, React 19, TypeScript, Postgres on Neon, Vercel free tiers) that turns a YouTube playlist into a course: timestamped notes, practice problems and a quiz that jumps the video to the moment that explains a wrong answer. The live library has 4 courses and 206 lessons, 171 with AI-written notes.
- Worked around YouTube blocking caption downloads from cloud servers (5 of 5 requests failed from Vercel) by splitting the work by where each step can run: the site stores metadata and marks lessons "transcript pending", and my Mac downloads captions and uploads them through an agent API.
- Stopped AI notes that pass format checks but teach nothing (an early run produced 37 lessons from one template) with a server-side quality gate: timestamps must match real transcript lines, quiz options cannot be copied from the summary, and sentences cannot repeat across lessons. Each lesson is written in a fresh agent session so cost stays linear.
- Built authentication without a library: argon2id passwords with lockout, 15-minute JWT access tokens in httpOnly cookies, and refresh tokens stored only as SHA-256 hashes and rotated on every use. Replaying an old refresh token revokes every session. Checked with 13 scripted scenarios.
- Gave AI agents safe write access to production through a custom MCP server (8 tools) over a token-scoped agent API with no delete routes. Tokens are shown once, stored hashed and revocable, and agent saves pass the same quality gate.

Portfolio Ask AI | *Full-Stack LLM Feature – React + FastAPI* | izsk.dev | github.com/sk-izsk/zsk-ai-resume-api

- Built a chat that answers recruiter questions straight from my resume: a React 19 / TypeScript / Vite frontend and a Python FastAPI backend on Render calling Groq's llama-3.3-70b-versatile.
- Grounded answers with a prompt pipeline that returns JSON only (`answer`, `sources`, `blocked`) and blocks off-topic questions.
- Kept inference cheap with Upstash Redis caching on versioned keys, IP rate limiting (3 requests per IP per hour) with 429 handling in the UI, and a `compact_profile()` step that sends reduced context instead of the full 54KB JSON.
- Handled Render free-tier cold starts with a `/health` pre-flight call and a loading overlay before the first `POST /chat`.

Whisper Chat | *Full-Stack Real-Time Messaging App* | Live Demo | github.com/sk-izsk/react-native-fullstack-whisper-chat

- Built real-time chat on web (React 19 / Vite) and mobile (React Native / Expo) over one shared Express 5 / Socket.IO / MongoDB TypeScript backend, with Clerk auth on all three.
- Kept cache invalidation separate from socket handling by splitting state: TanStack Query for server state, Zustand for socket connection, unread markers, typing indicators and presence.
- Added optimistic message sending, Sentry on mobile and modular socket logic, with the same behaviour on web and native without duplicating business logic.

Z-Graph | *Interactive Math Visualization Platform* | z-graph.vercel.app | github.com/sk-izsk/z-graph

- Built a multi-mode graphing app in React 18 / TypeScript covering 17 categories of math (algebra, calculus, polar, parametric, vector fields, Fourier series, Julia and Mandelbrot fractals, 3D implicit surfaces) with live formula editing, sliders and animation.
- Combined Mafs for 2D and Three.js / React Three Fiber for 3D on a shared Zustand layer, with mathjs parsing formulas and Web Workers keeping fractal sampling and surface meshing off the main thread.

ZTable Web | *Interactive Chemistry App* | zsk-periodic.vercel.app | github.com/sk-izsk/zsk-periodic

- Moved equation balancing into a Web Worker, lazy-loaded routes through TanStack Router and memoized the Zustand selectors behind the 118 periodic table cells, so interactions don't trigger broad re-renders.
- Built 8 features with React 19, TypeScript, Vite and Bun, including 3D atoms (Three.js / React Three Fiber), molar mass parsing, solubility lookup, PDF worksheet export and i18n.

Productify | *Full-Stack Product Management App* | izsk.dev

- Built a React 19 front end (Vite, React Router 7, compound components) on a typed Express 5 REST API with Drizzle ORM and PostgreSQL (Neon), TanStack Query for cache-aware mutations, and Clerk for authentication and role-based authorization.

EDUCATION

Concordia University Montréal, QC
M.Eng., Electrical & Computer Engineering *Sept 2016 – Dec 2018*

- Completed coursework in Computer Architecture Design, Microprocessor Applications, Digital Signal Processing, Model-Driven Software Engineering, Formal Hardware Verification, Real-Time Applications, Robotics.

DecodeMTL Montréal, QC
Full-Stack Web Development Bootcamp *April 2019 – May 2019*

- Completed a full-stack web program covering JavaScript, React, Node.js, Express, MongoDB, HTML5 and CSS3.